is to be deleted it is necessary to have a flag set in the record. Then, in a subsequent reading process, that record is skipped over.

Choice of file organisation

Serial file organisation is well suited to batch processing or for backing up data on magnetic tape. A direct access file is used if rapid access to an individual record in a large file is required. An example would be on a system with many users. In this case, the file that is used to check passwords when users log in should be direct-access. A sequential file is suitable for applications when multiple records are required from one search of the file. An example could be a family history file where a search could be used for all records with a particular family name.

At this point it is worth mentioning the difference between key fields in a file and primary keys in a database table. In the database table the primary key values must all be unique. This is not a requirement for key fields in any type of file. It may be sensible in certain applications to ensure key fields have unique values, but it is not mandatory.

16.03 Real numbers

A real number is one with a fractional part. When we write down a value for a real number in the denary system we have a choice. We can use a simple representation, or we can use an exponential notation (sometimes referred to as scientific notation). For example, the number 25.3 might be written as:

$$.253 \times 10^{2} \text{ or } 2.53 \times 10^{1} \text{ or } 25.3 \times 10^{0} \text{ or } 253 \times 10^{-1}$$

For this number, the simple expression is best. But if a number is very large or very small the exponential notation is the only sensible choice.

Floating-point and fixed-point representations

A binary code must be used for storing a real number in a computer system. One possibility is to use a fixed-point representation. In fixed-point representation, an overall number of bits is chosen with a defined number of bits for the whole number part and the remainder for the fractional part. The alternative is a **floating-point representation**. The format for a floating-point number can be generalised as:

$$\pm M \times R^{E}$$

In floating-point representation a defined number of bits are used for what is called the significand or mantissa, ±M. The remaining bits are used for the exponent or exrad, E. The radix, R is not stored in the representation; R has an implied value of 2.



KEY TERM

Floating-point representation: a representation of real numbers that stores a value for the mantissa and a value for the exponent

A simple example can be used to illustrate the differences between the two representations. Let's consider that a real number is to be stored in eight bits.

For the fixed-point option, a possible choice would be to use the most significant bit as a sign bit and the next five bits for the whole number part. This would leave two bits for the fractional part. Some important non-zero values in this representation are shown in Table 16.01. (The bits are shown with a gap to indicate the implied position of the binary point.)

Description	Binary code	Denary equivalent
Largest positive value	011111 11	31.75
Smallest positive value	000000 01	0.25
Smallest magnitude negative value	100000 01	−0.25
Largest magnitude negative value	111111 11	−31.75

Table 16.01 Example fixed-point representations (using sign and magnitude)

A possible choice for a floating-point representation would be four bits for the mantissa and four bits for the exponent with each using two's complement representation. The exponent is stored as a signed integer. The mantissa has to be stored as a fixed-point real value. The question now is where the binary point should be.

Two of the options for the mantissa being expressed in four bits are shown in Table 16.02(a) and Table 16.02(b). In each case, the denary equivalent is shown, and the position of the implied binary point is shown by a gap. Table 16.02(c) shows the three largest magnitude positive and negative values for integer coding that will be used for the exponent.

a

First bit pattern for a real value	Real value in denary
011 1	3.5
011 0	3.0
010 1	2.5
101 0	−3.0
100 1	−3.5
100 0	−4.0

b

Second bit pattern for a real value	Real value in denary
0 111	0.875
0 110	0.75
0 101	0.625
1 010	−0.75
1 001	−0.875
1 000	−1.0

c

Integer bit pattern	Integer value in denary
0111	7
0110	6
0101	5
1010	−6
1001	−7
1000	−8

Table 16.02 Coding a floating-point real value in eight bits (four for the mantissa and four for the exponent)

When the mantissa has the implied binary point immediately following the sign bit, a smaller spacing is produced between the values that can be represented. This is the preferred option for a floating-point representation. Using this option, the most important non-zero values for the floating-point representation are shown in Table 16.03. (The implied binary point and the mantissa exponent separation are shown by a gap.)

Description	Binary code	Denary equivalent
Largest positive value	0 111 0111	$0.875 \times 2^{7} = 112$
Smallest positive value	0 001 1000	$0.125 \times 2^{-8} = 1/2048$
Smallest magnitude negative value	1 111 1000	$-0.125 \times 2^{-8} = -1/2048$
Largest magnitude negative value	1 000 0111	$-1 \times 2^{7} = -128$

Table 16.03 Example floating-point representations

The comparison between the values in Tables 16.01 and 16.03 illustrate the greater range of positive and negative values available if floating-point representation is used.

Extension Question 16.02

a Using the methods suggested in Chapter 1 (Section 1.01) can you confirm for yourself that the denary equivalents of the binary codes shown in Tables 16.02 and Table 16.03 are as indicated?

b Can you also confirm that conversion from positive to negative (or the conversion from negative to positive) for a fixed-format real value still follows the rules defined in Chapter 1 (Section 1.02) for two's complement representation?

Precision and normalisation

You have to decide about the format of a floating-point representation in two respects. You have to decide the total number of bits to be used and decide on the split between those representing the mantissa and those representing the exponent. In practice, a choice for the total number of bits to be used will be available as an option when the program is written. However, the split between the two parts of the representation will have been determined by the floating-point processor. If you did have a choice you would base your decision on the fact that increasing the number of bits for the mantissa would give better precision for a value stored but would leave fewer bits for the exponent, which reduces the range of possible values.

To achieve maximum precision you have to normalise a floating-point number. (This normalisation is unrelated to the process associated with designing a database.) Precision increases with an increasing number of bits for the mantissa, so optimum precision will only be achieved if full use is made of these bits. In practice, that means using the largest possible magnitude for the value represented by the mantissa.

To illustrate this, we can consider the eight-bit representation used in Table 16.03. Table 16.04 shows possible representations for denary 2 using this representation.

Denary representation	Floating-point binary representation
0.125×2^4	0 001 0100
0.25×2^3	0 010 0011
0.5×2^2	0 100 0010

Table 16.04 Alternative representations of denary 2 using four bits each for mantissa and exponent

For a negative number we can consider representations for –4 as shown in Table 16.05.

Denary representation	Floating-point binary representation
-0.25×2^4	1 110 0100
-0.5×2^3	1 100 0011
-1.0×2^2	1 000 0010

Table 16.05 Alternative representations of denary –4 using four bits each for mantissa and exponent

When the number is represented with the highest magnitude for the mantissa, the two most significant bits are different. This fact can be used to recognise that a number is in a normalised representation. The values in Tables 16.03 and 16.04 also show how a number

could be normalised. For a positive number, the bits in the mantissa are shifted left until the most significant bits are 0 followed by 1. For each shift left the value of the exponent is reduced by 1.

The same process of shifting is used for a negative number until the most significant bits are 1 followed by 0. In this case, no attention is paid to the fact that bits are falling off the most significant end of the mantissa.

Extension Question 16.03

Look at the data in Table 16.03. Do you see any conflict with the above discussion? What is likely to be the approach in a typical floating-point system?

Conversion of representations

In Chapter 1 (Section 1.01), a number of methods for converting numbers into different representations were discussed. These only considered integer values. We now need to consider the conversion of real numbers.

We can start by considering the conversion of a simple real number, such as 4.75, into a simple fixed-point binary representation. This looks easy because 4 converts to 100 in binary and .75 converts to .11 in binary so the binary version of 4.75 should be:

100.11

However, remember that a positive number should start with 0. Can we just add a sign bit? For a positive number we can. Denary 4.75 can be represented as 0100.11 in binary.

For negative numbers we still want to use two's complement form. So, to find the representation of –4.75 we can start with the representation for 4.75 then convert it to two's complement as follows:

0100.11 converts to 1011.00 in one's complement

then to 1011.01 in two's complement

To check the result, we can apply Method 2 from Worked Example 1.01 in Chapter 1. 1011 is the code for –8 + 3 and .01 is the code for .25; –8 + 3 + .25 = –4.75.

We can now consider the conversion of a denary value expressed as a real number into a floating-point binary representation. Before considering the conversion method it should be remembered that most fractional parts do not convert to a precise representation. This is because the binary fractional parts represent a half, a quarter, an eighth, a sixteenth and so on. Unless a denary fraction is a sum of a collection of these values, there cannot be an accurate conversion. In particular, of the values from .1 through to .9, only .5 converts accurately. This was mentioned in Chapter 1 (Section 1.03) in the discussion about storing currency values.

The method for conversion of a positive value is as follows.

1. Convert the whole-number part using the method described in Chapter 1 (Section 1.01).
2. Add the 0 sign bit.
3. Convert the fractional part choosing a method from one of the examples in Worked Example 16.01.
4. Combine the whole number and fractional parts and enter these into the most significant of the bits allocated for the representation of the mantissa.